

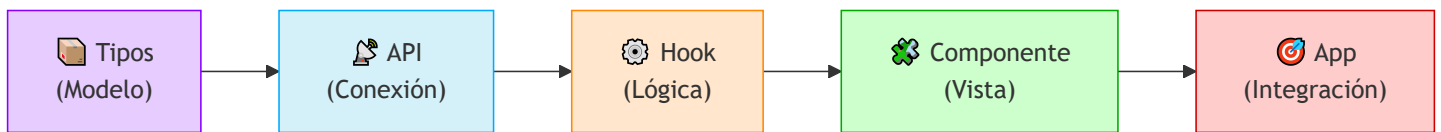
# **Guía Didáctica: Crear la Consulta de Empleados - Paso a Paso**

## **Objetivo de la Guía**

Vamos a construir la funcionalidad de "LISTAR EMPLEADOS" paso a paso:

1. Definir los TIPOS (modelo de datos)
2. Crear el CLIENTE HTTP (conexión con el backend)
3. Crear los ENDPOINTS (peticiones a la API)
4. Crear el HOOK PERSONALIZADO (lógica de negocio)
5. Crear el COMPONENTE LISTA (vista)
6. Integrar en APP (padre)

## **Paso 0: ¿Qué necesitamos?**





# Paso 1: Definir los Tipos (El Modelo)

## ¿Por qué empezar aquí?

LOS TIPOS SON EL "MOLDE" DE TUS DATOS

- ◆ Definen qué datos existen y de qué tipo son
  - ◆ Son la base que usará el resto del código
  - ◆ TypeScript los usa para evitar errores
- ⚠ Si los tipos están mal, todo el código tendrá errores  
Por eso es lo PRIMERO que debemos hacer.

## Archivo a crear: src/types/employee.types.ts

### Tipos de Empleado

#### Employee

---

- id: string
- cedula: string
- nombres: string
- salario: number
- departamento: string
  - activo: boolean
- createdAt: string
- updatedAt: string

#### EmployeeListResponse

---

- count: number
- data: Employee[]

#### EmployeeFormState

---

- cedula: string
- nombres: string
- salario: string
- departamento: string
- activo: boolean

## Explicación de cada tipo:

| Tipo                        | Propósito                                    | ¿Dónde se usa?     |
|-----------------------------|----------------------------------------------|--------------------|
| <b>Employee</b>             | Datos de un empleado (respuesta del backend) | Componentes, Hooks |
| <b>EmployeeListResponse</b> | Respuesta del backend al listar              | API.getAll()       |
| <b>EmployeeFormState</b>    | Datos del formulario (frontend)              | EmployeeForm       |

## Qué debe tener el archivo:

```
// 📁 src/types/employee.types.ts

// 1. Tipo Employee - El que viene del backend
export interface Employee {
  id: string;
  cedula: string;
  nombres: string;
  salario: number;
  departamento: string;
  activo: boolean;
  createdAt: string;
  updatedAt: string;
}

// 2. Tipo EmployeeListResponse - Respuesta al listar
export interface EmployeeListResponse {
  count: number;
  data: Employee[];
}

// 3. Tipo EmployeeFormState - Estado del formulario (frontend)
export interface EmployeeFormState {
  cedula: string;
  nombres: string;
  salario: string; // ← string porque es un input
  departamento: string;
  activo: boolean;
}
```

## ✅ Verificación del Paso 1:

# 1. Crear la carpeta types

```
mkdir src/types
```

# 2. Crear el archivo

```
touch src/types/employee.types.ts
```

# 3. Copiar el contenido de arriba

# 4. Verificar que no hay errores

```
npx tsc --noEmit
```

## 🔑 Paso 2: Crear el Cliente HTTP

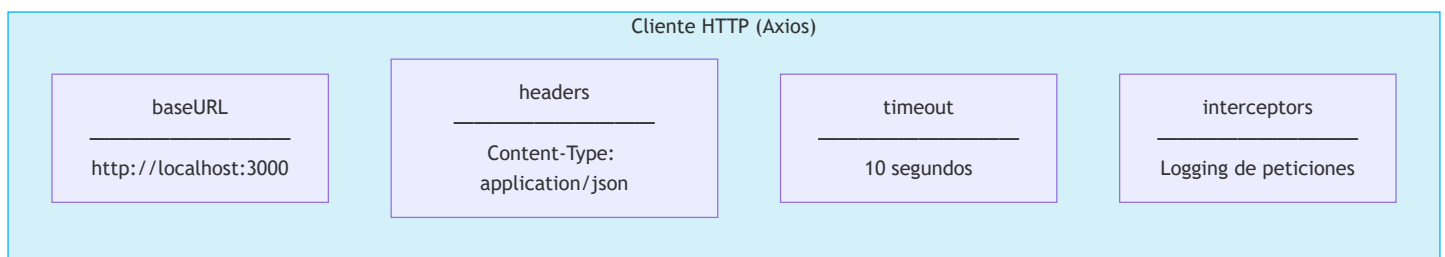
### ¿Por qué necesitamos un cliente HTTP?

AXIOS ES EL "MENSAJERO" DE TU APP

- ◆ Se encarga de comunicarse con el backend
- ◆ Hace las peticiones HTTP (GET, POST, PUT, DELETE)
- ◆ Maneja errores de red
- ◆ Configura headers y timeout

📌 Sin Axios, no podemos conectar con el backend

### Archivo a crear: src/api/client.ts



# Qué debe tener el archivo:

```
// 📁 src/api/client.ts

import axios from 'axios';

// URL del backend (desde .env o por defecto)
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000';

// Crear el cliente Axios
export const apiClient = axios.create({
  baseURL: API_BASE_URL,      // ← Dirección del backend
  headers: {
    'Content-Type': 'application/json', // ← Enviamos JSON
  },
  timeout: 10000,             // ← 10 segundos máximo de espera
});

// Interceptors: para ver qué peticiones hacemos (logging)
apiClient.interceptors.request.use((config) => {
  console.log(`🔗 ${config.method?.toUpperCase()} ${config.url}`);
  return config;
});

apiClient.interceptors.response.use(
  (response) => {
    console.log(`✅ ${response.status} ${response.config.url}`);
    return response;
  },
  (error) => {
    console.error(`❌ Error: ${error.message}`);
    return Promise.reject(error);
  }
);
```



## Verificación del Paso 2:

# 1. Crear la carpeta api

```
mkdir src/api
```

# 2. Crear el archivo

```
touch src/api/client.ts
```

# 3. Instalar Axios si no está

```
npm install axios
```

# 4. Copiar el contenido de arriba

# 5. Verificar

```
npx tsc --noEmit
```



## Paso 3: Crear los Endpoints

### ¿Qué son los endpoints?

ENDPOINTS = LAS "RUTAS" DE COMUNICACIÓN

- ◆ Cada endpoint es una función que hace una petición HTTP
- ◆ Define qué endpoint del backend llamar
- ◆ Envía los datos y recibe las respuestas
- ◆ Convierte la respuesta en un objeto tipado

📌 Los endpoints son la "capa de traducción" entre tu frontend y el backend.

**Archivo a crear:** `src/api/employee.api.ts`



## API de Empleados

getAll()

GET /api/employees  
→ Employee[]

getById(id)

GET /api/employees/:id  
→ Employee

create(data)

POST /api/employees  
→ Employee

update(id, data)

PUT /api/employees/:id  
→ Employee

delete(id)

DELETE  
/api/employees/:id  
→ void

# Qué debe tener el archivo:

```
// 📁 src/api/employee.api.ts

import { apiClient } from './client';
import type {
  Employee,
  EmployeeListResponse,
} from '../types/employee.types';

const API_URL = '/api/employees';

export const EmployeeAPI = {
  /**
   * GET /api/employees
   * Listar todos los empleados
   */
  async getAll(): Promise<Employee[]> {
    // 1. Hacer la petición GET
    const response = await apiClient.get<EmployeeListResponse>(API_URL);

    // 2. Extraer solo la lista de empleados (response.data.data)
    // porque la respuesta es: { count: 10, data: [...] }
    return response.data.data;
  },

  /**
   * GET /api/employees/:id
   * Obtener un empleado por ID
   */
  async getById(id: string): Promise<Employee> {
    const response = await apiClient.get<{ data: Employee }>(
      `${API_URL}/${id}`
    );
    return response.data.data;
  },
};
```

## ✅ Verificación del Paso 3:

# 1. Crear el archivo

```
touch src/api/employee.api.ts
```

# 2. Copiar el contenido de arriba

# 3. Verificar que los imports son correctos

```
npx tsc --noEmit
```

## ⚙️ Paso 4: Crear el Hook Personalizado

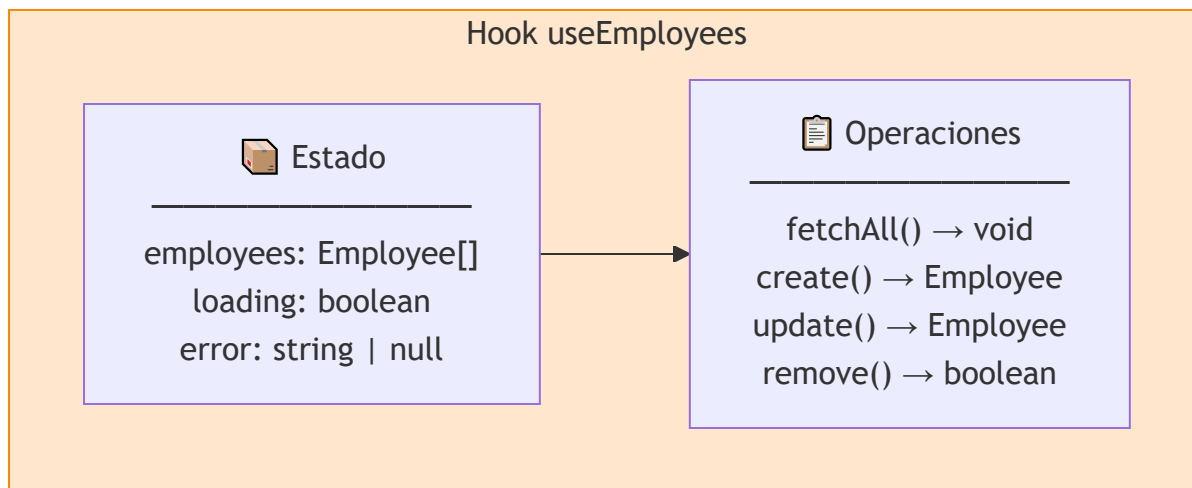
### ¿Qué es un hook personalizado?

HOOK = LA "LÓGICA DE NEGOCIO" DE TU APP

- ◆ Contiene el estado de los datos
- ◆ Contiene las funciones para obtenerlos
- ◆ Se comunica con la API
- ◆ Provee los datos y funciones a los componentes

📌 El hook es el "controlador" que conecta la vista con los datos.

**Archivo a crear:** `src/hooks/useEmployees.ts`



# Qué debe tener el archivo (SOLO LISTAR):

```
// 📁 src/hooks/useEmployees.ts
```

```
import { useState, useEffect, useCallback } from 'react';
import { EmployeeAPI } from '../api/employee.api';
import type { Employee } from '../types/employee.types';

/**
 * Extrae un mensaje de error de forma segura
 */
function getErrorMessage(error: unknown): string {
  if (typeof error === 'object' && error !== null) {
    const err = error as { response?: { data?: { error?: string; message?: string } } };
    if (err.response?.data?.error) return err.response.data.error;
    if (err.response?.data?.message) return err.response.data.message;
  }
  if (error instanceof Error) return error.message;
  if (typeof error === 'string') return error;
  return 'Ocurrió un error inesperado';
}

/**
 * Hook para gestionar empleados
 */
export function useEmployees() {
  // =====
  // ESTADO
  // =====
  const [employees, setEmployees] = useState<Employee[]>([]); // ← Lista vacía al inicio
  const [loading, setLoading] = useState<boolean>(true);      // ← Cargando al inicio
  const [error, setError] = useState<string | null>(null);    // ← Sin error al inicio

  // =====
  // FUNCIÓN PARA OBTENER EMPLEADOS
  // =====
  const fetchAll = useCallback(async () => {
    setLoading(true);      // ← Empezamos a cargar
    setError(null);        // ← Limpiamos errores anteriores

    try {
      const data = await EmployeeAPI.getAll(); // ← Llamamos a la API
      setEmployees(data); // ← Guardamos los datos
    }
  }, []);

  // Llamamos a la función para obtener empleados al inicio
  fetchAll();

  // Efecto para actualizar la lista de empleados cuando cambie el estado de carga
  useEffect(() => {
    if (!loading) {
      fetchAll();
    }
  }, [loading]);
}
```

```

    } catch (err: unknown) {
      const message = getErrorMessage(err);    // ← Extraemos el mensaje
      setError(message);    // ← Guardamos el error
      console.error('Error fetching employees:', err);
    } finally {
      setLoading(false);    // ← Terminamos de cargar (éxito o error)
    }
  }, []);

  // =====
  // CARGAR AUTOMÁTICAMENTE AL INICIO
  // =====
  useEffect(() => {
    fetchAll();    // ← Cuando el componente se monta, carga los datos
  }, []);    // ← Dependencias vacías = solo una vez

  // =====
  // RETORNAR DATOS Y FUNCIONES
  // =====
  return {
    employees,
    loading,
    error,
    fetchAll,    // ← Por si queremos recargar manualmente
  };
}

```

## Verificación del Paso 4:

# 1. Crear la carpeta hooks

```
mkdir src/hooks
```

# 2. Crear el archivo

```
touch src/hooks/useEmployees.ts
```

# 3. Copiar el contenido de arriba

# 4. Verificar

```
npx tsc --noEmit
```

# Paso 5: Crear el Componente Lista

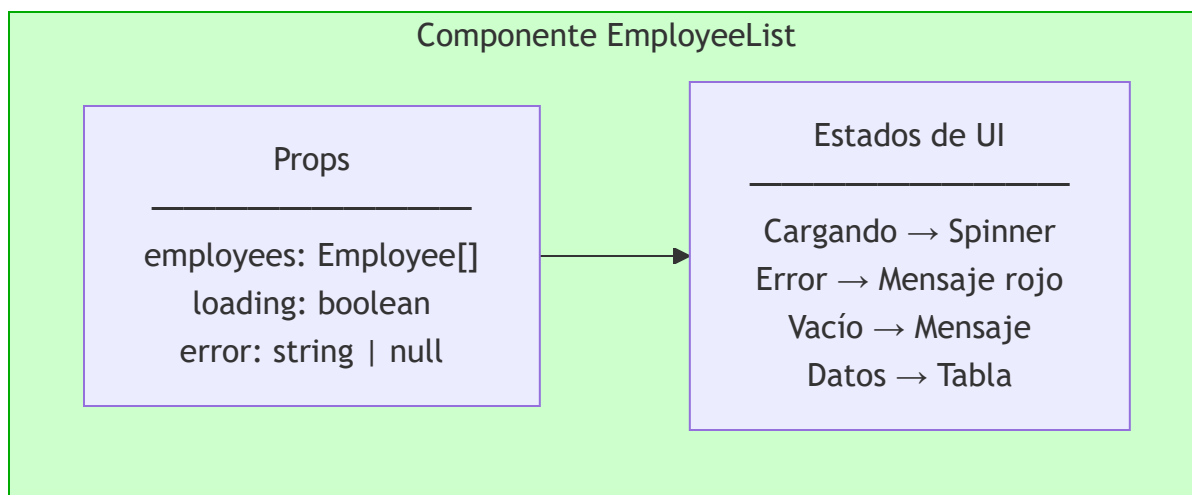
## ¿Qué es el componente lista?

COMPONENTE LISTA = LA "VISTA" DE LOS DATOS

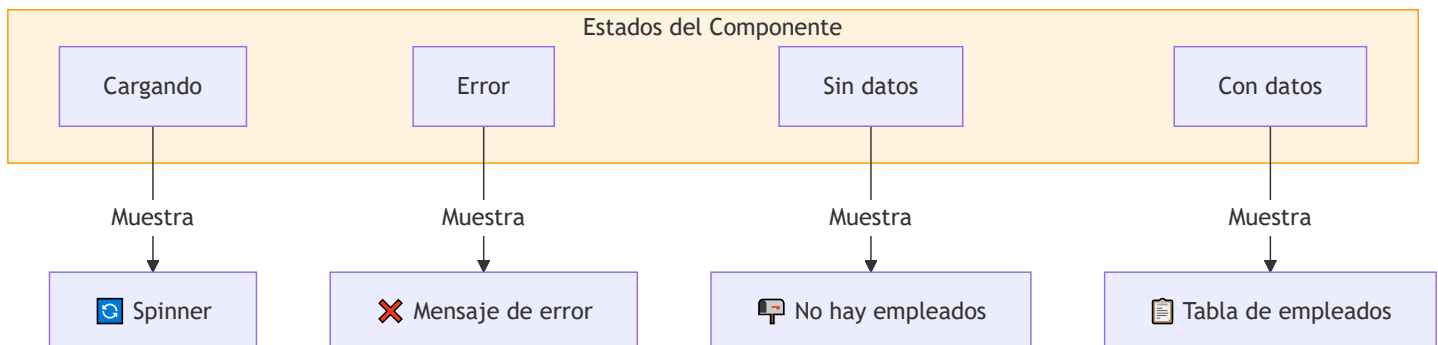
- ◆ Muestra los empleados en una tabla
- ◆ Muestra un spinner mientras carga
- ◆ Muestra un mensaje si hay error
- ◆ Muestra "No hay datos" si la lista está vacía

🚩 Es un componente "tonto": solo recibe props y las muestra.

**Archivo a crear:** `src/components/EmployeeList.tsx`



## Diseño del Componente



# Qué debe tener el archivo:

```
// 📁 src/components/EmployeeList.tsx

import React from 'react';
import type { Employee } from '../types/employee.types';

// =====
// PROPS QUE RECIBE
// =====
interface EmployeeListProps {
  employees: Employee[];      // ← Lista de empleados
  loading: boolean;           // ← ¿Está cargando?
  error: string | null;       // ← ¿Hay error?
}

// =====
// COMPONENTE
// =====
export const EmployeeList: React.FC<EmployeeListProps> = ({
  employees,
  loading,
  error,
}) => {
  // =====
  // CASO 1: CARGANDO
  // =====
  if (loading) {
    return (
      <div className="flex justify-center items-center h-64">
        <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600">
        </div>
      </div>
    );
  }

  // =====
  // CASO 2: ERROR
  // =====
  if (error) {
    return (
      <div className="bg-red-50 border border-red-400 text-red-700 px-4 py-3 rounded">
        ❌ Error: {error}
      </div>
    );
  }
}
```

```

    );
}

// =====
// CASO 3: SIN DATOS
// =====
if (employees.length === 0) {
    return (
        <div className="text-center py-12 bg-gray-50 rounded-lg">
            <p className="text-gray-500 text-lg">No hay empleados registrados</p>
            <p className="text-gray-400 text-sm">
                Haz clic en "Nuevo Empleado" para agregar uno
            </p>
        </div>
    );
}

// =====
// CASO 4: CON DATOS → TABLA
// =====
return (
    <div className="overflow-x-auto shadow-md rounded-lg">
        <table className="min-w-full bg-white">
            {/* Cabecera */}
            <thead className="bg-gray-100 border-b">
                <tr>
                    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">#</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Cédula</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Nombres</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Salario</th>
                    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Estado</th>
                </tr>
            </thead>

```







## Verificación del Paso 5:

# 1. Crear la carpeta components

```
mkdir src/components
```

# 2. Crear el archivo

```
touch src/components/EmployeeList.tsx
```

# 3. Copiar el contenido de arriba

# 4. Verificar

```
npx tsc --noEmit
```



## Paso 6: Integrar en App (El Padre)

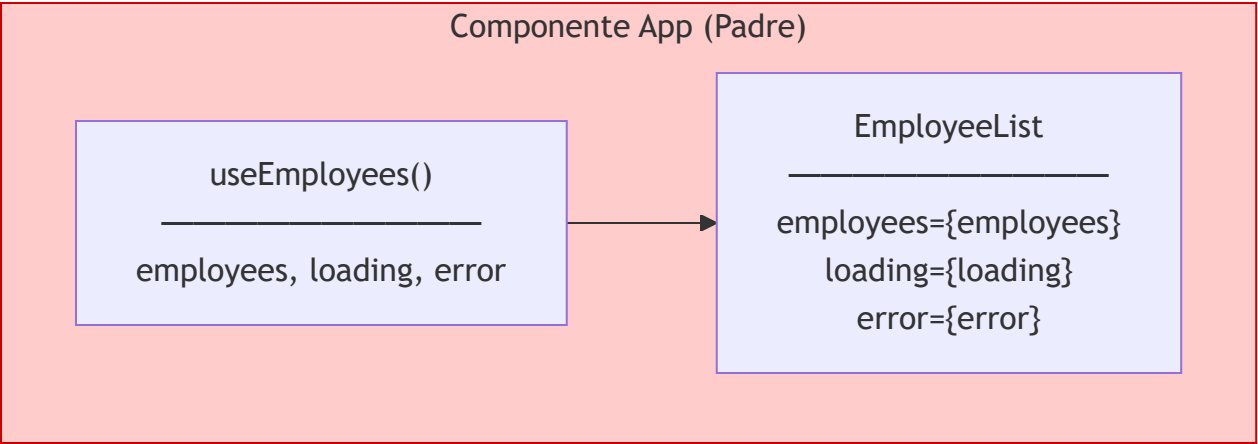
### ¿Qué es App?

APP = EL "ORQUESTADOR" DE LA APP

- ◆ Es el componente PADRE de todos
- ◆ Usa el hook para obtener los datos
- ◆ Pasa los datos al componente lista
- ◆ Controla el formulario y el modal

📌 App coordina todo y conecta las piezas.

**Archivo a modificar:** src/App.tsx



# Qué debe tener el archivo:

```
// 📁 src/App.tsx

import React from 'react';
import { useEmployees } from '../hooks/useEmployees';
import { EmployeeList } from '../components/EmployeeList';

function App() {
  // =====
  // 1. OBTENER DATOS DEL HOOK
  // =====
  const {
    employees,    // ← Lista de empleados
    loading,      // ← ¿Está cargando?
    error,        // ← ¿Hay error?
  } = useEmployees();

  // =====
  // 2. RENDERIZAR
  // =====
  return (
    <div className="min-h-screen bg-gray-50">
      {/* Navbar */}
      <nav className="bg-white shadow-sm border-b">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4">
          <h1 className="text-2xl font-bold text-gray-900">
            👤 Gestión de Empleados
          </h1>
        </div>
      </nav>

      {/* Contenido principal */}
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        {/* ✅ Pasar datos al componente lista */}
        <EmployeeList
          employees={employees}
          loading={loading}
          error={error}
        />
      </main>
    </div>
  );
}
```

```
}
```

```
export default App;
```

## ✓ Verificación del Paso 6:

# 1. Modificar src/App.tsx con el contenido de arriba

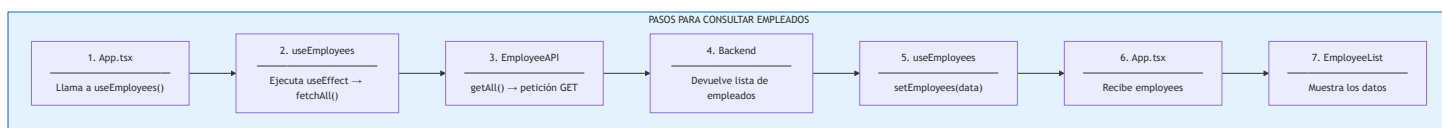
# 2. Verificar que no hay errores

```
npx tsc --noEmit
```

# 3. Iniciar el proyecto

```
npm run dev
```

## 📊 Resumen Visual: Flujo de Datos





# Checklist de Verificación Final

# 1. Verificar que todos los archivos existen

```
ls src/types/employee.types.ts
ls src/api/client.ts
ls src/api/employee.api.ts
ls src/hooks/useEmployees.ts
ls src/components/EmployeeList.tsx
ls src/App.tsx
```

# 2. Verificar que no hay errores de TypeScript

```
npx tsc --noEmit
```

# 3. Verificar que el backend está corriendo

```
curl http://localhost:3000/api/employees
```

# 4. Iniciar el frontend

```
npm run dev
```

# 5. Abrir en el navegador

```
# http://localhost:5173
```



## Preguntas de Reflexión

1. ¿Por qué empezamos definiendo los tipos?

Respuesta: Porque son el "molde" de los datos y todo el código depende de ellos.

2. ¿Por qué el hook se llama "useEmployees" y no "getEmployees"?

Respuesta: Porque use indica que es un hook de React (puede usar otros hooks como useState, useEffect, etc.).

3. ¿Por qué EmployeeList no tiene lógica de negocio?

Respuesta: Para que sea reutilizable y fácil de probar. Solo recibe props y las muestra.

4. ¿Por qué App es el que usa el hook y no EmployeeList?

Respuesta: Porque App es el padre que orquesta los datos. EmployeeList solo muestra.



# Resumen del Flujo

## FLUJO COMPLETO

1. App.tsx se monta
2. useEmployees() se ejecuta
3. useEffect() → fetchAll()
4. EmployeeAPI.getAll() → Petición HTTP
5. Backend responde con datos
6. setEmployees(data) → Actualiza el estado
7. React re-renderiza App
8. App pasa datos a EmployeeList
9. EmployeeList muestra la tabla